2D Graphic Lib concept

Legend:



Framebuffer



BumpBox



FramebufferList

Refs:

- micropython\extmod\modframebuf.c
- micropython\examples\natmod\framebuf\framebuf.c"

Canvas:



Canvas attributes:

- Background Color
- List of DirtyRegions

- The color of the canvas, is defined as transparent.
- Transparent pixels are transparent for collision also.
- The canvas is data root. All objects are added to this canvas.
- The canvas owns a framebuffer, which represents the display in size and color model.
- The canvas owns a list, which represents the dirty-regions.
- Every change to the owned objects updates the dirty-regions list.
- The canvas, or parts of it, is sent to the display every time-tick.The dirty-regions list determines what is sent to the display
- The dirty-regions are organized in a list; We have an add-to function that decides, whether to add the region as new data, or to merge it to an existing one;
- The canvas owns the global coordinate-system.
- The objects owned by the canvas are organized in layers. The layer determines the order in which the objects are rendered.
- The canvas has an animation-timer for auto animate the sprite objects.

Sprites:

Sprite attributes:

- Sprite Id
- Sprite Name
- Layer
- Position x
- Position y
- HasChanged flag
- Scale
- BumpBox
- Framebuffer list
- Index to current FB
- Auto Animate Flag
- Rotation
- ~~View Port~~

- A Sprite is an object, owned by the canvas.
- A Sprite has a global coordinate system and a viewport cs. There are methods to convert between them.
- A Sprite has a „changed“ flag. It becomes true, if the FB-index has changed;
- When a sprite moves, rotates or changes FB, we update the canvas dirty-region;
- A Sprite is drawn to a layer.
- Sprites can collide only within the same layer.
- The BumpBox defines the area relevant to collisions.
- The BumpBox rotates, with the sprites rotation.
- The sprite owns a list of one or more frames, used for animation, or morphing.
- A frame-list entry points to a framebuffer. This framebuffer is the unmodified blueprint.
- (ViewPort and BumpBox is the same)
- The Framebuffers of the sprites can have different color models, like 565, 8-Bit, or even 4-bit. There are transfer functions to map them into the canvas.

Framebuffers:

Framebuffer attributes:

- Framebuffer Id
- Framebuffer Name
- ColorModel
- Width
- Height

- We can draw into a FB, with all primitive functions, like pixel, line, rectangle, etc.;
- We can load images from disk into the FB.
- Framebuffers can be organized in sets. Sprites can point to this sets.

Framebuffersets:

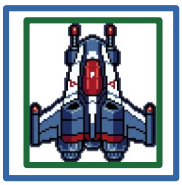
FramebufferSet attributes:

- Set Id
- Set Name
- Contained FBs count
- ColorModel

A FramebufferSet is a collection of framebuffers, which are all of the same size and ColorModel.

- Several Sprites can point to a single FbList.
- The images in the container are unchanged, unrotated, unscaled blueprints.

Simple Sprite:



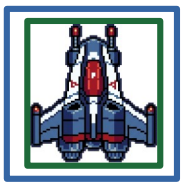
Sprite attributes:

- Id
- Name
- Layer
- HasChanged flag
- Scale
- ColorModel
- BumpBox

- A Simple Sprite is the basic sprite object.
- It's framebuffer is tied to the object.
- For simple objects that move and collide.

Animated Sprite:

#0

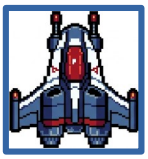


Sprite attributes:

- Id
- Name
- Layer
- HasChanged flag
- Scale
- ColorModel
- BumpBox
- FramebufferSet
- Index to current FB in the set
- AutoAnimate Flag

- An Animated Sprite has no own FB, but a pointer to a FB list.
- The current FB index, determines, which image will be displayed.
- The AutoAnimate flag increments the current FB index automatically every animation-tick.
- The minimum width and height of the FB is calculated with association to the FbList.
- The sprite has increment and decrement functions to cycle through the Fbs.
- We can enable auto-inc and tie the animation to a global timer.
- With disabled auto-inc we set the displayed FB manually.
- The sprite can change the FB-set, the new set can be smaller or larger.

#0



#1



#2



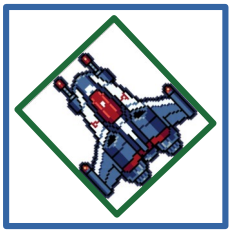
#3



The FramebufferSet, the Sprite points to;

Rotated Sprite:

#0

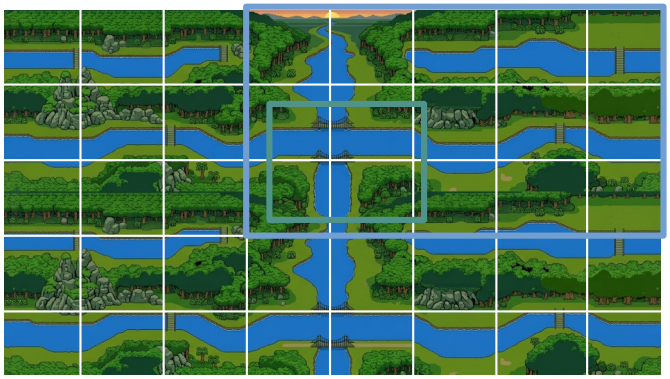


Attributes:

- Id
- Name
- Layer
- HasChanged flag
- Scale
- ColorModel
- BumpBox
- Framebuffer list
- Index to current FB
- Auto Animate Flag
- Rotation

- A Rotated Sprite is the extension of an Animated Sprite.
- The minimum size of the FB is calculated with association to the FbList, the diagonal of the image is used.
- The BumpBox rotates with the image.
- The rotation pivot is the center of the FB.
- ~~Sprite has a pivot-point (origin).~~
- ~~The origin can be set, even outside the sprite.~~
- ~~Default position of the origin is in the center.~~

Map Sprites and Tile Loading:



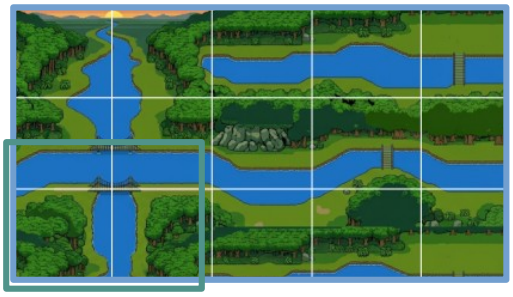
- A Map Sprites FB is much larger than the canvas.
- Collision detection between Map and a Sprite on the same layer works pixelwise only. No collision is detected, for the canvas color, which represents transparency.
- A Map Sprite has no rotation.
- The BumpBox attribute determines the viewport.
- The map has, like all Sprites a local coordinate system.
- ToDo: Do we need to load tiles to the map? Other parts must then be scrolled.

Attributes:

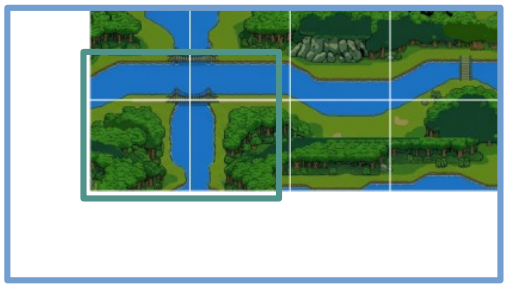
- Id
- Name
- Layer
- HasChanged flag
- Scale
- ColorModel
- BumpBox

Tile Loading:

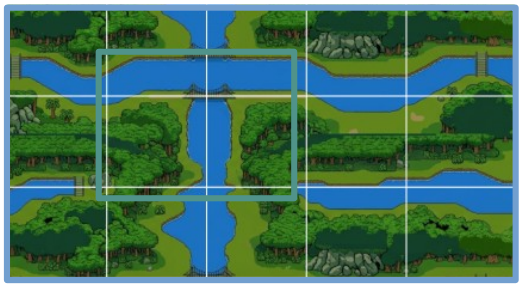
- A set of tiles is loaded to the Map Sprites FB (blue frame). The parts outside the blue frame are stored on disk in tiles.
- The ViewPort is taken to the display (green frame)
- The ViewPort is moved to animate, for example, a flight over a landscape.



- When the ViewPort moves out of the bounds of the FB, the tile loading mechanism is triggered.



- The ViewPort is tranformed one tilesize.



- The new tiles (or all tiles, respectively a new set of tiles) are loaded into the framebuffer.

Collision Detection:

Um in einer Ebene (2D) zu bestimmen, ob ein Punkt rechts oder links von einem Vektor (bzw. einer Geraden in Blicksrichtung des Vektors) liegt, nutzt man die Determinante (oder das 2D-Kreuzprodukt). Das Verfahren (2D-Determinante)

Gegeben seien:

Ein Richtungsvektor $\vec{v} = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$

Ein Startpunkt (oder Ursprung) der Vektors $\vec{a} = \begin{pmatrix} a_x \\ a_y \end{pmatrix}$

Ein zu prüfender Punkt $P = \begin{pmatrix} p_x \\ p_y \end{pmatrix}$

Man berechnet den Vektor vom Startpunkt zum Punkt P: $\vec{w} = P - \vec{a} = \begin{pmatrix} p_x - a_x \\ p_y - a_y \end{pmatrix}$.

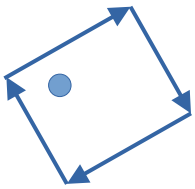
Nun berechnet man die Determinante der beiden Vektoren $\vec{v}$ und $\vec{w}$ (oder die z-Komponente des 3D-Kreuzprodukts):

$$\det(\vec{v}, \vec{w}) = v_x \cdot w_y - v_y \cdot w_x = v_x \cdot (p_y - a_y) - v_y \cdot (p_x - a_x)$$

Auswertung des Ergebnisses:

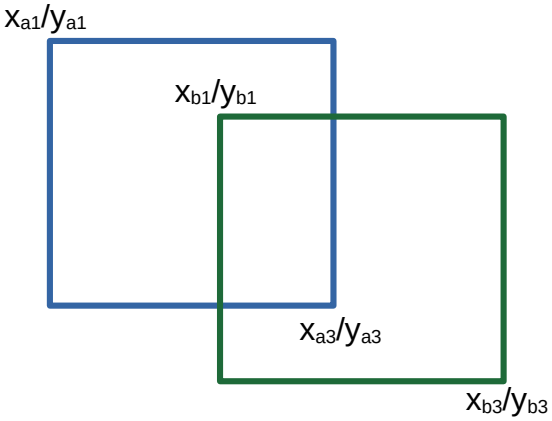
det > 0 (positiv): Der Punkt liegt links vom Vektor.
det < 0 (negativ): Der Punkt liegt rechts vom Vektor.
det = 0: Der Punkt liegt exakt auf der Geraden (kollinear zum Vektor).

(Hinweis: „Links“ und „rechts“ beziehen sich dabei immer auf die Blickrichtung vom Startpunkt in Richtung der Vektorspitze).



- Liegt ein Punkt rechts von allen vier Vektoren, liegt er innerhalb des Rechtecks.

Bounding Box Collision Detection:



x_start = max(xA1, xB1)
y_start = max(yA1, yB1)
x_end = min(xA2, xB2)
y_end = min(yA2, yB2)

x_start < x_end und y_start < y_end, existiert ein gültiger Schnittbereich.

Andernfalls (wenn x_start >= x_end oder y_start >= y_end) überschneiden sich die Boxen nicht (Schnittfläche = 0)

```
def get_box_intersection(boxA, boxB):
    xA = max(boxA[0], boxB[0])
    yA = max(boxA[1], boxB[1])
    xB = min(boxA[2], boxB[2])
    yB = min(boxA[3], boxB[3])

    if xA < xB and yA < yB:
        return (xA, yA, xB, yB) # Schnittbox
    return None # Kein Schnittpunkt
```